

SmartSearchify: AI-Enabled Semantic Search for eSankhyiki Portal

Rahul Kichar
23110264
IIT Gandhinagar
Gandhinagar, Gujarat, India
23110264@iitgn.ac.in

Vadithya Harsha Vardhan
Nayak
23110349
IIT Gandhinagar
Gandhinagar, Gujarat, India
23110349@iitgn.ac.in

Sailaja Mudu
23110288
IIT Gandhinagar
Gandhinagar, Gujarat, India
23110288@iitgn.ac.in

Arvind Banoth
23110058
IIT Gandhinagar
Gandhinagar, Gujarat, India
23110058@iitgn.ac.in

Raj Kumar Gupta
23110267
IIT Gandhinagar
Gandhinagar, Gujarat, India
23110267@iitgn.ac.in

ABSTRACT

The eSankhyiki Portal provides centralized access to India's official statistics and plays a key role in promoting data-driven decision making across various sectors. However, the portal currently relies on manually navigated dropdown menus for querying data, which can be unintuitive and inefficient especially for non-expert users. We chose this project from the problem statements provided by "Ministry of Statistics and Programme Implementation". This project addresses this usability issue by proposing the integration of an AI-powered semantic search system. The system allows users to enter queries in natural language, which are then mapped intelligently to the appropriate statistical indicators and filters using Natural Language Processing (NLP) and machine learning models. This eliminates the need for technical knowledge of database structure and indicator naming conventions. By leveraging NLP techniques and training models on user-query patterns, the system improves accessibility, usability, and discoverability of data. We demonstrate the approach by building a prototype that maps queries to relevant database fields and significantly enhances user experience. This solution can enable wider sections of society, from researchers to policymakers, to efficiently access and analyze national statistical data.

Keywords: Semantic Search, Natural Language Processing, eSankhyiki Portal, AI-based Query Mapping, Data Accessibility.

1. Introduction

India's growing digital infrastructure aims to make official data more accessible to its citizens. One such effort is the eSankhyiki Portal, which aggregates statistical indicators from various government departments. Although the portal provides structured and verified datasets, its user interface is heavily reliant on dropdown-based filtering mechanisms. This makes it difficult for users especially those without prior domain knowledge to locate the specific data they are looking for. Often, users must know exact indicator names or follow a rigid hierarchy of filters to access relevant information. This limits the portal's effectiveness for the public, journalists, students, and even some policymakers. To address this issue, our project proposes the development of a semantic search system powered by Artificial Intelligence. Instead of relying on dropdowns, users can input queries in simple natural language, such as "female literacy rate in Rajasthan in 2020". The system interprets the query using Natural Language Processing (NLP) techniques and maps it to the correct indicators and filters in the database. This improves the discoverability of data while also enhancing the overall user

experience. By applying machine learning and NLP, this system reduces the cognitive load on users and bridges the gap between human language and structured data. Our approach not only makes the eSankhyiki Portal more user-friendly but also ensures inclusive access to public information.

2. Preliminaries

The eSankhyiki Portal consolidates India's official statistics but suffers from complex dropdown navigation and technical jargon. To improve accessibility, this project introduces a semantic search system allowing users to query in natural language.

It applies NLP techniques like entity recognition, tokenization, lemmatization, and sentence embeddings (all-MiniLM-L6-v2) to represent text meaningfully. User queries are first classified (e.g., CPI, NAS, PLFS) using LinearSVC, followed by a hybrid search: initial filtering through cosine similarity and refined ranking via a cross-encoder model (ms-marco-MiniLM-L-6-v2). This balances system speed and accuracy, even with large datasets.

3. Related Work

Several prior efforts inform this project:

1. Semantic Search moves beyond keyword matching with tools like Haystack and ElasticSearch.
2. NLIDB (Natural Language Interfaces to Databases) enables plain language queries in systems like Google's NLP tools and Power BI Q&A.
3. Transformer-based Embeddings (BERT, RoBERTa, sentence-transformers) improve query-document matching by capturing semantic relationships.
4. Hybrid Search Methods use quick similarity checks combined with deeper reranking models (e.g., Hugging Face and Facebook AI's FAISS).
5. Government Initiatives like India's Open Government Data and the UK's ONS are enhancing public data access but still largely rely on keyword-based searches.
6. By adapting these methods to the eSankhyiki Portal, the project aims to make statistical data retrieval faster, simpler, and more inclusive for all users.

4. Data Preparation and Conceptual Foundations

The development of the semantic search system for the eSankhyiki Portal followed a structured, data science-driven workflow. The process focused on transforming natural language queries into meaningful, structured database queries using advanced NLP and machine learning techniques.

4.1 Data Collection:

To enable accurate semantic mapping of user queries to statistical indicators, the first step involved collecting and organizing the relevant data from the eSankhyiki portal.

- **Web Scraping:** We used tools like BeautifulSoup, Selenium, and Requests, along with a Chrome extension called SingleFile, to extract structured metadata about statistical tables. Using SingleFile, we first saved complete HTML pages of the eSankhyiki portal, and then employed BeautifulSoup (bs4) for HTML parsing to retrieve the necessary information.

Each record in the dataset corresponds to a unique statistical indicator and includes the following attributes:

- Title
- Product
- Category
- Geography
- Frequency
- Description and ID
- Reference Period
- Release Data
- Table Number
- Download URL
- Data Source
- search_text

The search_text column is the core implementation in this dataset. It was created by concatenating the Title and Description fields of each entry. This provided a richer textual context, which allowed the NLP model to semantically match user queries with the most relevant datasets.

4.2 Data Enhancement

To enhance the dataset and improve the semantic matching of user queries, the following steps were taken:

- **Text Rephrasing:** Using libraries like spaCy, NLTK, and WordNet, the title and description fields of the dataset were rephrased. This rephrasing process ensured the system could better match user queries, even when phrased differently from the original dataset entries.
- **Linguistic Processing:** The additional use of NLP libraries helped expand the dataset by rewording and generating alternative phrasings for each sentence, ensuring broader query coverage.
- **Database Structuring:** All these fields were compiled into a structured dataset that served as the foundation for indexing, query matching, and training the NLP models.

The search_text column played a crucial role in enabling semantic similarity between user queries and records.

4.3 Data Preprocessing

To prepare user queries for semantic matching, a comprehensive preprocessing pipeline was implemented using established NLP libraries. Each step played a critical role in refining the input and improving the system's understanding of query intent.

1. **Tokenization:** The first step in preprocessing was tokenizing the user queries using spaCy. This involved breaking down the text into individual tokens or words. Tokenization is essential for analyzing the structure of the query.
2. **Normalization:**
 - **Lowercasing:** All tokens were converted to lowercase using spaCy and NLTK to ensure uniformity. This prevented the system from treating the same word differently based on capitalization (e.g., "India" vs "india").
 - **Punctuation Removal:** All punctuation marks (such as commas, periods, and question marks) were removed using spaCy and NLTK, as they do not contribute significantly to the semantic content of the query.
3. **Stopword Removal:** Words that do not carry significant meaning, such as "the", "is", "and", or "of", were filtered out using predefined stopword lists available in both spaCy and NLTK.

Removing stopwords ensured that the system focused on the most informative terms within a query.

4. **Lemmatization:** Lemmatization was performed using spaCy, which reduced words to their base or dictionary forms (e.g., "running" → "run", "categories" → "category"). This process helped normalize different morphological forms of words and improved semantic similarity detection.
5. **Synonym Expansion:** To further enhance the query interpretation, synonyms were generated using WordNet via the NLTK interface. This step enabled the system to recognize and match different words with similar meanings (e.g., "purchase" and "buy") thereby increases its flexibility in understanding diverse user expressions.

4.4 Semantic Matching and Similarity Search

- **Entity Recognition:** Using named entity recognition (NER), the system identified key elements such as Geography (e.g., "Rajasthan"), Reference period (e.g., "2020"), and domain-specific terms like "unemployment rate" or "female literacy".
- **Vector Embeddings:** To semantically interpret user queries and match them with relevant statistical entries, we transformed both the user input and dataset search_text fields into numerical representations called vector embeddings. These embeddings capture the meaning and context of text in a format that machine learning models can understand. We used the all-MiniLM-L6-v2 model from Hugging Face Transformers, which is known for its balance of performance and efficiency. This model encodes each sentence into a 384-dimensional dense vector. In this 384-dimensional space, semantically similar texts such as a user query and a dataset description are placed closer together. This allowed the system to capture deeper semantic meanings beyond keyword matches.
- **Cosine Similarity:** Cosine similarity measures the cosine of the angle between two vectors in a high-dimensional space. It is used to calculate the similarity between the query vector and dataset vectors. The query and dataset entries are transformed into vectors using NLP techniques like word embeddings. Cosine similarity is then calculated to measure the degree of similarity between the query and dataset vectors, ranges from 0 to 1.

For example: A query like "female literacy in Rajasthan in 2020" and a dataset entry describing the same topic will have vectors with high cosine similarity.

A higher cosine similarity score indicates a greater similarity in meaning between the query and dataset entry.

5. Methodology

5.1 Before the user enters the query:

- **Setting up the environment:** We used a web Framework called Streamlit to make our application user-friendly and easy to access. Streamlit allows us to quickly create a web page where users can search the query and see the results interactively. In addition, the app uses some memory tracking tools like psutil and tracemalloc to keep track of the memory the application uses. This helps us monitor the app to ensure it is efficient and does not slow down over time.

- **Loading the data and models:** before the user enters its query, the application needs some essential resources, which are the following:
 - **Dataset:** A preprocess CSV file containing Information about various data sets (title, description, product name, title embedding, search text, search text embedding, download link, etc.). This file already contains the embedding for search, which saves query search time. We don't need to compute the embeddings while searching.
 - **Classifier Model:** The application needs to load a machine learning-based classifier called "LinearSVC", which classifies the query into its product (like 'ASI', 'CAMS', etc.). This helps the application determine which product field the user is interested in.
 - **Sentence Embedder:** This application uses a language model, "all-MiniLM-L6-v2", which converts the user query into embedding very quickly, which helps in comparing the query with existing catalogues.
 - **Cross Encoder:** This powerful AI model reaches and reranks the results more accurately through deep semantic search. For this task, we used "ms-marco-MiniLM-L-6-v2".
- **Preparing the Data:** the data set contains the embedding stored as a string. Hence, we convert the strings back into proper numeric arrays and can use them for similarity comparison letters. We use search text embedding to concatenate the data set title and description.

5.2 After entering the query:

The user enters using the search created using stimulate, and after that following process happens:

Classification of query: First, we convert the query into embeddings using sentence embeddings and then try to classify the query into its most relevant product to find out which field of information the user is interested in. The linear support vector classifier returns the probability of each product for a query. For further processes, we pick up the top 3 products with the highest probabilities. The top 3 products also have an associated confidence score (probability).

Performing a deep semantic search:

- **Cosine similarity filtering:** First, the cosine similarity between the user query and old catalogue entries within The Chosen products is calculated. mathematical way, how close pieces of text are in the context of meaning. it picks the top 50 most similar entries per product.
- **Cross-encoder reranking:** To get more accuracy, a cross-encoder model is used to compare the query and each of the 50 selected entries deeply. The cross-encoder assigns a new score to each matching. Finally, it returns the top few results in the following way: three results for the product with the highest probability, two for the second highest probability product and one for the third highest probability product.
- The purpose of this filtering is to enhance search time. Cosine similarity is very quick to calculate, and deep semantic search takes comparatively more time. That's why, in the first search, we quickly filtered out the top 50 queries using cosine similarity. Then, we

used encoder for deep semantic searches to get a more accurate response. This approach significantly enhances the search Speed while maintaining accuracy.

- In addition to product-specific search, the app performs a General semantic search across all the products to find the top 10 matches overall — the same approach for this. Also, instead of classifying into the products, we directly apply cosine similarity over all the search embeddings and filter out the top 50 embeddings. Then, a cross-encoder was used for deep semantic search, which gave the top 10 results.

Here, we are doing two searches: the first is a specific search, and the second is an overall search. This dual makes sure that the user is getting the most appropriate results.

5.3 Displaying results:

After processing the query, the app displays the search results' intersections

- Each of the top predicted products groups product-specific results. It also shows the confidence percentage for each product.
- Overall top matches: The second column shows the best results across the entire catalogue.
- By clicking on any of the results, we can see the whole description results, loading the critical Information such as Title, Product Name, Category, Geograph, Frequency (how often the data is updated), Reference Period (timeframe the data covers), Release Date, Table Number, Data Source, Description, A Download Link, we can click on the result to expand and view all the details in an organised way. We can download the data set in an Excel file by clicking on the download link.

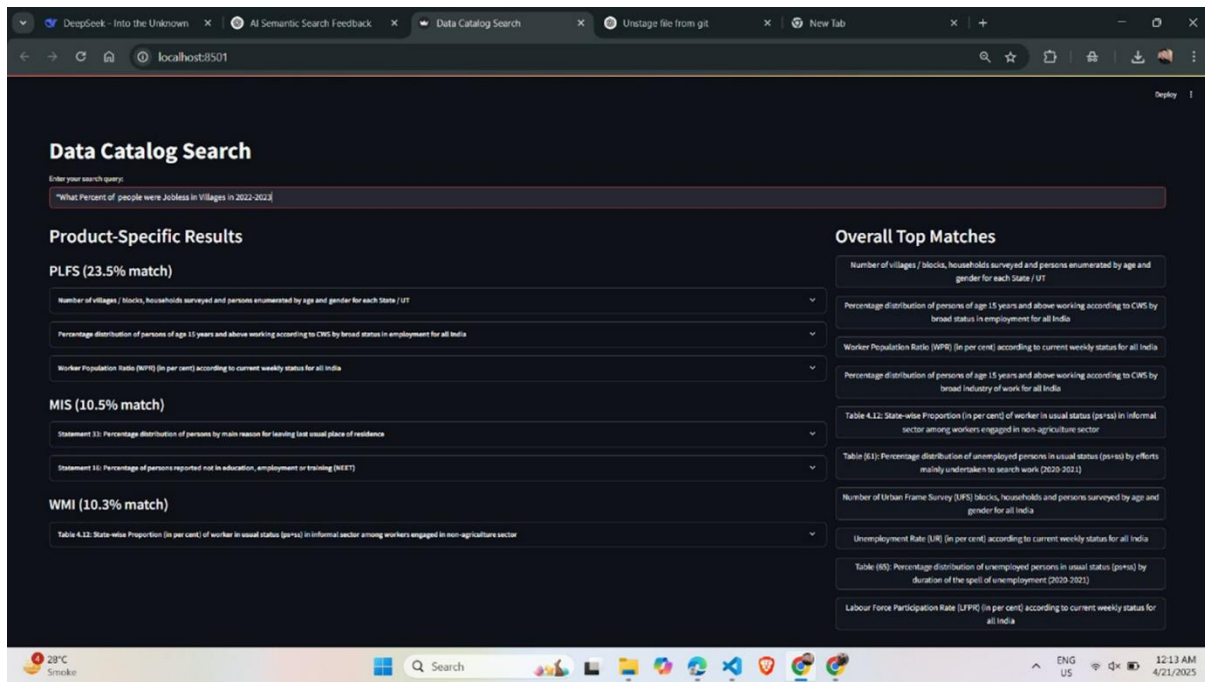
5.4 Managing state and improving speed:

This application uses session state for unnecessary calculations. It means the app doesn't redo the search if the user repeats a query or doesn't change it. Results are also remembered between page refreshers as long as the session is active. The application also tracks the time taken for each query and prints out the memory usage for debugging purposes.

5.5 Memory and efficient tracking:

Memory consumption and performance timing throughout the application life cycle.

- RSS Memory Usage: Physical memory being used by the app.
- Tracemalloc Stats: Memory being tracked within Python.
- This helps the developers to ensure that the application is memory efficient and can efficiently handle many queries without grace.



6. Experiments

6.1 Classification model selection:

For the classification task, the goal was to predict the relevant product category (e.g., CPI, NAS, PLFS) from a given data set of catalogue embeddings. For the speed ride, several machine-learning models were used. we avoided using neural networks because we didn't want to be a black box.

Model	Accuracy	Model Size (KB)	Avg Inference Time (ms)
LogisticRegression	1.000000	30.975586	1.675526
LinearSVC	1.000000	30.846680	1.332919
RandomForest	1.000000	4342.696289	30.631145
KNN	1.000000	8584.832031	38.093090
XGBoost	0.999298	880.937500	2.170642

The data set was well classified into products. In each product, the data set catalogues are particularly related to one field only, That's why our models give 100% accuracy. LinearSVC achieved 100% accuracy with a tiny size (~31 KB) and fast inference time (~1.33 ms per batch), making it ideal for real-time applications. In contrast, random forest and KNN models have high memory and computational costs. Hence, linear SVC was selected due to its excellent balance of accuracy and efficiency.

6.2 Search model selection:

Compare several Transformer based models. I tried these models on a small Corpus of query. The tested models included BERT (base), Roberta, several sentence-transformer models, and cross-encoders. Unlike sentence transformers that embed query and document independently, CrossEncoders jointly encode query-document pairs and better capture their relationship. being slide slower, the cross encoder delivers a more precise ranking, especially for our case where meaning alignment was crucial. Hence, we adopted a hybrid search first, reducing the results to 50 by filtering out using cosine similarity danger and then reranking the top 50 using the crossing coder.

7. Results

From the classification experiments, the linear SVC emerged as the best model with an excellent balance of accuracy, efficiency and simplicity. Handled multiple class classifications across 10 product categories with perfect precision and minimal time consumption.

In the semantic search benchmarking, combining Cosine Similarity for initial filtering and cross-encoder/ms-marco-MiniLM-L-6-v2 for the Re ranking offers the best balance between accuracy and precision. The cosine similarity handles the processing task very fast(~20 ms), and the cross-encoder ensures that the final top results match the user query accurately. The selected models are integrated into the final streamlit web application. This application texts a query that performs a unique search and displays the best matching data set grouped by the product and overall score. The interactive interference shows improvement information for each result loading download link and description. Some of the tracking results are the following:

- RAM requirement for first-time loading: ~1.1GB.
- RAM requirement for query search: 200 MB.
- Inference time: ~45 ms.

8. Conclusion:

The project successfully demonstrates how modern NLP techniques can enhance the search and discoveries in structured data catalogues. By leveraging sentence embedding, classifying models, and cross-encoding, the system understands user intent beyond the keyword and gives highly relevant results. Careful live model testing shows that the selected components are accurate but also efficient and production-ready. The final application is scalable, accurate and user-friendly, bringing intelligence to efficient data Discovery.

9. References

- <https://huggingface.co/>
- <https://scikit-learn.org/stable/modules/generated/sklearn.svm.LinearSVC.html>
- <https://www.sbert.net/> (sentence transformer)
- <https://www.esankhyiki.gov.in/>
- <https://github.com/rahulkhichar7/SmartSearchify-AI-Enabled-Semantic-Search-for-eSankhyiki-Portal.git>